



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

SEARCH STRATEGIES IN AI

Informed and Uninformed Search

Lab 4 Report

Submitted By:

Name: **Aditya Kumar Shah**

Roll No: **080BCT011**

Date: **2083-03-23**

Submitted To:

Department of Electronics
and Computer Engineering

Institute of Engineering,
Pulchowk Campus

Subject: Artificial Intelligence

Contents

1	Objectives	4
2	Theoretical Background	4
2.1	Search Problem Formulation	4
2.2	Uninformed (Blind) Search	4
2.3	Informed (Heuristic) Search	5
2.4	Breadth-First Search (BFS)	5
2.5	Uniform-Cost Search (UCS)	5
2.6	Depth-First Search (DFS)	6
2.7	Iterative-Deepening DFS (IDDFS)	6
2.8	Greedy Best-First Search	6
2.9	A* Search	7
2.10	The Romanian Map Problem	7
3	Program 1 — Breadth-First Search (BFS)	9
3.1	Problem Statement	9
3.2	Algorithm	9
3.3	Python Implementation	9
3.4	Output	10
3.5	Discussion	10
4	Program 2 — Uniform-Cost Search (UCS)	11
4.1	Problem Statement	11
4.2	Algorithm	11
4.3	Python Implementation	11
4.4	Output	12
4.5	Discussion	12
5	Program 3 — Depth-First Search (DFS)	13
5.1	Problem Statement	13
5.2	Algorithm	13
5.3	Python Implementation	13
5.4	Output	14
5.5	Discussion	14
6	Program 4 — Iterative-Deepening DFS (IDDFS)	15
6.1	Problem Statement	15
6.2	Algorithm	15
6.3	Python Implementation	15
6.4	Output	16
6.5	Discussion	16
7	Program 5 — Greedy Best-First Search	17
7.1	Problem Statement	17
7.2	Algorithm	17
7.3	Python Implementation	17

7.4	Output	18
7.5	Discussion	18
8	Program 6 — A* Search	19
8.1	Problem Statement	19
8.2	Algorithm	19
8.3	Python Implementation	19
8.4	Output	20
8.5	Discussion	20
9	Program 7 — Comparison of All Search Algorithms	21
9.1	Problem Statement	21
9.2	Output	21
9.3	Discussion	22
10	Conclusion	23

1 Objectives

1. To understand the classification of search strategies into **uninformed** (blind) and **informed** (heuristic) search.
2. To implement **Breadth-First Search (BFS)** and analyse its completeness and optimality.
3. To implement **Uniform-Cost Search (UCS)** and understand how path cost drives expansion.
4. To implement **Depth-First Search (DFS)** and observe its memory efficiency vs. non-optimality.
5. To implement **Iterative-Deepening DFS (IDDFS)** and appreciate its combination of BFS completeness with DFS memory efficiency.
6. To implement **Greedy Best-First Search** and understand how heuristic guidance can be fast but suboptimal.
7. To implement **A* Search** and verify its optimality when an admissible heuristic is used.
8. To compare all six algorithms on the Romanian Map problem and draw conclusions.

2 Theoretical Background

2.1 Search Problem Formulation

A **search problem** is defined by the following components:

Table 1: Search Problem Components

Component	Description
Initial State	The state from which the search begins (e.g., Arad)
Actions	Set of actions available from a state (e.g., move to neighbouring city)
Transition Model	Describes the result of an action: $\text{Result}(s, a) = s'$
Goal Test	Determines if the goal state has been reached (e.g., Bucharest)
Path Cost	A function assigning a numeric cost to each path (e.g., sum of road distances)

2.2 Uninformed (Blind) Search

Uninformed search strategies use **no additional information** about the problem domain beyond the problem definition. They can only distinguish goal from non-goal states and generate successors.

Key Characteristics:

- No heuristic or domain knowledge is used

- Systematic exploration of the state space
 - May be inefficient for large or complex problems
 - Includes: BFS, UCS, DFS, IDDFS
-

2.3 Informed (Heuristic) Search

Informed search strategies use **problem-specific knowledge** beyond the problem definition to find solutions more efficiently. This knowledge is encoded in a **heuristic function** $h(n)$ that estimates the cost of the cheapest path from node n to a goal node.

Key Characteristics:

- Uses a heuristic function $h(n)$ to guide search
 - Typically explores fewer nodes than uninformed search
 - Quality of search depends on quality of the heuristic
 - Includes: Greedy Best-First Search, A* Search
-

2.4 Breadth-First Search (BFS)

BFS expands the **shallowest unexpanded node** first, using a **FIFO queue** (first-in, first-out). It explores all nodes at depth d before any nodes at depth $d + 1$.

Properties:

- **Complete:** Yes (if branching factor is finite)
 - **Optimal:** Yes (if all step costs are equal); No (if step costs differ)
 - **Time Complexity:** $O(b^d)$ where b = branching factor, d = depth of shallowest goal
 - **Space Complexity:** $O(b^d)$ — must store all nodes at current depth
-

2.5 Uniform-Cost Search (UCS)

UCS expands the node with the **lowest path cost** $g(n)$ first, using a **priority queue** ordered by cumulative cost. It is BFS generalised to handle varying step costs.

Properties:

- **Complete:** Yes (if step costs ≥ 0)
 - **Optimal:** Yes (always finds the cheapest path for non-negative costs)
 - **Time Complexity:** $O(b^{1+\lceil C^*/\epsilon \rceil})$ where C^* is optimal cost, ϵ is minimum step cost
 - **Space Complexity:** $O(b^{1+\lceil C^*/\epsilon \rceil})$
-

2.6 Depth-First Search (DFS)

DFS expands the **deepest unexpanded node** first, using a **LIFO stack** (last-in, first-out). It plunges down a single branch until it reaches a dead end, then backtracks.

Properties:

- **Complete:** No (may get stuck in infinite branches); Yes in finite spaces with visited tracking
 - **Optimal:** No (may find a longer or more expensive path)
 - **Time Complexity:** $O(b^m)$ where m = maximum depth of the search tree
 - **Space Complexity:** $O(bm)$ — only stores nodes on current path
-

2.7 Iterative-Deepening DFS (IDDFS)

IDDFS repeatedly applies **depth-limited DFS** with increasing depth limits: 0, 1, 2, 3, ... It combines the **completeness of BFS** with the **memory efficiency of DFS**.

Properties:

- **Complete:** Yes (like BFS)
 - **Optimal:** Yes (if step costs are equal); No (if step costs differ)
 - **Time Complexity:** $O(b^d)$ — same as BFS (redundant work is dominated by deepest level)
 - **Space Complexity:** $O(bd)$ — same as DFS (only stores one branch)
-

2.8 Greedy Best-First Search

Greedy Best-First Search expands the node that appears to be **closest to the goal**, using only the heuristic function $h(n)$ to decide which node to expand next. It uses a priority queue ordered by $h(n)$.

Properties:

- **Complete:** No (can get stuck in loops); Yes with visited tracking in finite spaces
 - **Optimal:** No (greedily minimises $h(n)$, not actual path cost)
 - **Time Complexity:** $O(b^m)$ in worst case; often much better with a good heuristic
 - **Space Complexity:** $O(b^m)$ in worst case
-

2.9 A* Search

A* Search expands the node with the **lowest evaluation function** $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost to reach node n and $h(n)$ is the estimated cost from n to the goal. It combines the actual path cost (UCS) with the heuristic estimate (Greedy).

Properties:

- **Complete:** Yes (if branching factor is finite and $h(n)$ is admissible)
- **Optimal:** Yes (if $h(n)$ is **admissible** — never overestimates the true cost)
- **Time Complexity:** Depends on heuristic accuracy; $O(b^d)$ in worst case
- **Space Complexity:** $O(b^d)$ — stores all generated nodes

Admissible Heuristic: $h(n) \leq h^*(n)$ for all n , where $h^*(n)$ is the true cheapest cost from n to the goal. The straight-line distance is admissible because it can never exceed the actual road distance.

2.10 The Romanian Map Problem

The Romanian Map is a classic benchmark from *Artificial Intelligence: A Modern Approach* (Russell & Norvig). It models 20 Romanian cities connected by roads with distances as edge weights. The task is to find a route from **Arad** to **Bucharest**.

The **straight-line distance** (SLD) to Bucharest serves as an admissible heuristic for informed search algorithms:

Table 2: Straight-Line Distances to Bucharest (Heuristic $h(n)$)

City	$h(n)$	City	$h(n)$
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	RimnicuVilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

3 Program 1 — Breadth-First Search (BFS)

3.1 Problem Statement

Implement Breadth-First Search on the Romanian Map to find a route from Arad to Bucharest. BFS should find the shortest path in terms of number of hops.

3.2 Algorithm

1. Start
 2. Create an empty queue and a visited set
 3. Enqueue the start node with path = [start]; mark as visited
 4. While the queue is not empty:
 - (a) Dequeue the front element ($node, path$)
 - (b) If $node = goal$, return path
 - (c) For each unvisited neighbour of $node$ (in sorted order):
 - i. Mark neighbour as visited
 - ii. Enqueue ($neighbour, path + [neighbour]$)
 5. If queue empties without finding goal, return failure
 6. End
-

3.3 Python Implementation

```
1 def bfs(graph, start, goal):
2     visited = set()
3     queue = [(start, [start])]
4     visited.add(start)
5     exploration_order = []
6
7     while queue:
8         node, path = queue.pop(0)
9         exploration_order.append(node)
10
11         if node == goal:
12             return path, exploration_order
13
14         for neighbor in sorted(graph[node].keys()):
15             if neighbor not in visited:
16                 visited.add(neighbor)
17                 queue.append((neighbor, path + [neighbor]))
18
```

```
19      return None, exploration_order
```

Listing 1: Breadth-First Search

3.4 Output

```
=====
1. BREADTH-FIRST SEARCH (BFS)
=====
Path Found: Arad -> Sibiu -> Fagaras -> Bucharest
Path Cost:  450
Hops:      3
Nodes Explored: ['Arad', 'Sibiu', 'Timisoara', 'Zerind',
                 'Fagaras', 'Oradea', 'RimnicuVilcea', 'Lugoj', 'Bucharest']
```

Listing 2: BFS output

3.5 Discussion

BFS found the path Arad → Sibiu → Fagaras → Bucharest with a cost of 450. This is the shortest path in terms of **number of hops** (3 hops), but it is **not the cheapest path**. BFS is optimal only when all step costs are equal; since the Romanian Map has varying road distances, a cheaper 4-hop path exists (cost 418, found by UCS and A*). BFS explored 9 nodes before reaching the goal.

4 Program 2 — Uniform-Cost Search (UCS)

4.1 Problem Statement

Implement Uniform-Cost Search on the Romanian Map to find the cheapest route from Arad to Bucharest. UCS should find the optimal (minimum cost) path.

4.2 Algorithm

1. Start
 2. Create a priority queue ordered by path cost; initialise with $(0, start, [start])$
 3. Create an empty visited set
 4. While the priority queue is not empty:
 - (a) Pop the node with the lowest cumulative cost $(cost, node, path)$
 - (b) If $node$ is already visited, skip it
 - (c) Mark $node$ as visited
 - (d) If $node = goal$, return path and cost
 - (e) For each unvisited neighbour of $node$:
 - i. Calculate new cost $= cost + edge_cost$
 - ii. Push $(new_cost, neighbour, path + [neighbour])$ into the priority queue
 5. If priority queue empties without finding goal, return failure
 6. End
-

4.3 Python Implementation

```
1 import heapq
2
3 def ucs(graph, start, goal):
4     visited = set()
5     pq = [(0, start, [start])]
6     exploration_order = []
7
8     while pq:
9         cost, node, path = heapq.heappop(pq)
10
11         if node in visited:
12             continue
13         visited.add(node)
14         exploration_order.append(node)
15
```

```
16         if node == goal:
17             return path, cost, exploration_order
18
19         for neighbor, edge_cost in sorted(graph[node].items()):
20             if neighbor not in visited:
21                 heapq.heappush(pq,
22                               (cost + edge_cost, neighbor, path + [
23                                   neighbor]))
24
25     return None, float('inf'), exploration_order
```

Listing 3: Uniform-Cost Search

4.4 Output

```
=====
2. UNIFORM-COST SEARCH (UCS)
=====
Path Found: Arad -> Sibiu -> RimnicuVilcea -> Pitesti ->
           Bucharest
Path Cost: 418
Hops:      4
Nodes Explored: ['Arad', 'Zerind', 'Timisoara', 'Sibiu',
                 'Oradea', 'RimnicuVilcea', 'Lugoj', 'Fagaras', 'Mehadia',
                 'Pitesti', 'Craiova', 'Drobeta', 'Bucharest']
```

Listing 4: UCS output

4.5 Discussion

UCS found the optimal path Arad → Sibiu → RimnicuVilcea → Pitesti → Bucharest with a cost of **418**. This is the cheapest possible route. Unlike BFS which found a shorter-hop but more expensive path (450), UCS always expands the node with the lowest cumulative cost, guaranteeing optimality for non-negative edge costs. UCS explored 13 nodes — more than BFS — because it must explore all cheaper partial paths before confirming optimality.

5 Program 3 — Depth-First Search (DFS)

5.1 Problem Statement

Implement Depth-First Search (iterative, using a stack) on the Romanian Map to find a route from Arad to Bucharest. DFS should demonstrate its memory-efficient but potentially non-optimal behaviour.

5.2 Algorithm

1. Start
 2. Create a stack and push (*start*, [*start*])
 3. Create an empty visited set
 4. While the stack is not empty:
 - (a) Pop the top element (*node*, *path*)
 - (b) If *node* is already visited, skip it
 - (c) Mark *node* as visited
 - (d) If *node* = *goal*, return path
 - (e) For each unvisited neighbour of *node* (push in reverse sorted order so alphabetically first is explored first):
 - i. Push (*neighbour*, *path* + [*neighbour*]) onto the stack
 5. If stack empties without finding goal, return failure
 6. End
-

5.3 Python Implementation

```
1 def dfs(graph, start, goal):
2     visited = set()
3     stack = [(start, [start])]
4     exploration_order = []
5
6     while stack:
7         node, path = stack.pop()
8
9         if node in visited:
10             continue
11         visited.add(node)
12         exploration_order.append(node)
13
14         if node == goal:
```

```
15         return path, exploration_order
16
17     for neighbor in sorted(graph[node].keys(), reverse=True):
18         if neighbor not in visited:
19             stack.append((neighbor, path + [neighbor]))
20
21     return None, exploration_order
```

Listing 5: Depth-First Search

5.4 Output

```
=====
3. DEPTH-FIRST SEARCH (DFS)
=====
Path Found: Arad -> Sibiu -> Fagaras -> Bucharest
Path Cost: 450
Hops: 3
Nodes Explored: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest']
```

Listing 6: DFS output

5.5 Discussion

DFS found the path Arad → Sibiu → Fagaras → Bucharest with a cost of 450. While it happened to find the goal quickly (only 4 nodes explored), this is **not the optimal path**. DFS's path depends entirely on the order in which neighbours are pushed onto the stack — a different ordering could lead to a completely different (and potentially much longer or more expensive) path. DFS is memory-efficient ($O(bm)$) but provides **no guarantee of optimality**.

6 Program 4 — Iterative-Deepening DFS (IDDFS)

6.1 Problem Statement

Implement Iterative-Deepening DFS on the Romanian Map. IDDFS should combine the completeness of BFS with the memory efficiency of DFS by repeatedly applying depth-limited DFS with increasing depth limits.

6.2 Algorithm

1. Start
2. Set $depth = 0$
3. Repeat:
 - (a) Perform Depth-Limited Search (DLS) from start with current $depth$
 - (b) If goal found at this depth, return path
 - (c) Increment $depth = depth + 1$
 - (d) If $depth > max_depth$, return failure
4. End

Depth-Limited Search (DLS):

- (a) If $depth = 0$ and $node = goal$, return success
 - (b) If $depth > 0$:
 - i. For each unvisited neighbour:
 - A. Recursively call DLS with $depth - 1$
 - B. If found, propagate success back up
 - ii. Backtrack: remove neighbour from visited
 - (c) Return failure
-

6.3 Python Implementation

```
1 def dls(graph, node, goal, depth, visited, path):
2     if depth == 0 and node == goal:
3         return True, path
4     if depth > 0:
5         for neighbor in sorted(graph[node].keys()):
6             if neighbor not in visited:
7                 visited.add(neighbor)
8                 found, result = dls(graph, neighbor, goal,
9                                     depth - 1, visited, path + [neighbor])
```

```

10         if found:
11             return True, result
12             visited.discard(neighbor)
13     return False, None
14
15 def iddfs(graph, start, goal, max_depth=20):
16     exploration_order = []
17     for depth in range(max_depth + 1):
18         visited = {start}
19         found, path = dls(graph, start, goal, depth,
20                           visited, [start])
21         exploration_order.append(
22             f"Depth {depth}: visited {sorted(visited)}")
23         if found:
24             return path, depth, exploration_order
25     return None, -1, exploration_order

```

Listing 7: Iterative-Deepening DFS

6.4 Output

```

=====
4. ITERATIVE-DEEPENING DFS (IDDFS)
=====
Path Found: Arad -> Sibiu -> Fagaras -> Bucharest
Path Cost: 450
Hops: 3
Found at Depth: 3
Depth-by-depth exploration:
Depth 0: visited ['Arad']
Depth 1: visited ['Arad']
Depth 2: visited ['Arad']
Depth 3: visited ['Arad', 'Bucharest', 'Fagaras', 'Sibiu']

```

Listing 8: IDDFS output

6.5 Discussion

IDDFS found the path Arad → Sibiu → Fagaras → Bucharest at depth 3 with a cost of 450. Like BFS, it finds the shortest-depth path (3 hops) but **not the cheapest path**. The depth-by-depth trace shows that depths 0, 1, and 2 could not reach Bucharest — the goal was only found at depth 3. IDDFS's key advantage is its **linear space complexity** $O(bd)$, making it the preferred uninformed search method when memory is limited. The apparent redundancy of re-exploring shallower depths is acceptable because the number of nodes at the deepest level dominates.

7 Program 5 — Greedy Best-First Search

7.1 Problem Statement

Implement Greedy Best-First Search on the Romanian Map using the straight-line distance to Bucharest as the heuristic. It should expand the node that appears closest to the goal at each step.

7.2 Algorithm

1. Start
 2. Create a priority queue ordered by $h(n)$; initialise with $(h(\text{start}), \text{start}, [\text{start}])$
 3. Create an empty visited set
 4. While the priority queue is not empty:
 - (a) Pop the node with the smallest $h(n)$: $(h, \text{node}, \text{path})$
 - (b) If node is already visited, skip it
 - (c) Mark node as visited
 - (d) If $\text{node} = \text{goal}$, return path
 - (e) For each unvisited neighbour of node :
 - i. Push $(h(\text{neighbour}), \text{neighbour}, \text{path} + [\text{neighbour}])$ into the priority queue
 5. If priority queue empties without finding goal, return failure
 6. End
-

7.3 Python Implementation

```
1 import heapq
2
3 def greedy_best_first(graph, start, goal, heuristic):
4     visited = set()
5     pq = [(heuristic[start], start, [start])]
6     exploration_order = []
7
8     while pq:
9         h, node, path = heapq.heappop(pq)
10
11         if node in visited:
12             continue
13         visited.add(node)
14         exploration_order.append(node)
15
```

```

16         if node == goal:
17             cost = sum(graph[path[i]][path[i+1]]
18                         for i in range(len(path) - 1))
19             return path, cost, exploration_order
20
21         for neighbor in sorted(graph[node].keys()):
22             if neighbor not in visited:
23                 heapq.heappush(pq,
24                               (heuristic[neighbor], neighbor,
25                               path + [neighbor]))
26
27     return None, float('inf'), exploration_order

```

Listing 9: Greedy Best-First Search

7.4 Output

```

=====
5. GREEDY BEST-FIRST SEARCH
=====
Path Found: Arad -> Sibiu -> Fagaras -> Bucharest
Path Cost: 450
Hops: 3
Nodes Explored: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest']

```

Listing 10: Greedy Best-First output

7.5 Discussion

Greedy Best-First Search found the path Arad → Sibiu → Fagaras → Bucharest with a cost of 450. It greedily chose Sibiu ($h = 253$) over Zerind ($h = 374$) and Timisoara ($h = 329$) from Arad, then Fagaras ($h = 176$) over RimnicuVilcea ($h = 193$) and Oradea ($h = 380$) from Sibiu, and finally Bucharest ($h = 0$) from Fagaras. While it found the goal very quickly (only 4 nodes explored), the path is **not optimal** — it missed the cheaper route through RimnicuVilcea and Pitesti (cost 418) because $h(\text{Fagaras}) = 176 < h(\text{RimnicuVilcea}) = 193$, even though the RimnicuVilcea route is actually cheaper overall.

8 Program 6 — A* Search

8.1 Problem Statement

Implement A* Search on the Romanian Map using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual path cost and $h(n)$ is the straight-line distance heuristic. A* should find the optimal (cheapest) path.

8.2 Algorithm

1. Start
 2. Create a priority queue ordered by $f(n) = g(n) + h(n)$; initialise with $(h(\text{start}), 0, \text{start}, [\text{start}])$
 3. Create an empty visited set
 4. While the priority queue is not empty:
 - (a) Pop the node with the smallest $f(n)$: $(f, g, \text{node}, \text{path})$
 - (b) If node is already visited, skip it
 - (c) Mark node as visited
 - (d) If $\text{node} = \text{goal}$, return path and cost g
 - (e) For each unvisited neighbour of node :
 - i. Calculate $g' = g + \text{edge_cost}$
 - ii. Calculate $f' = g' + h(\text{neighbour})$
 - iii. Push $(f', g', \text{neighbour}, \text{path} + [\text{neighbour}])$ into the priority queue
 5. If priority queue empties without finding goal, return failure
 6. End
-

8.3 Python Implementation

```
1 import heapq
2
3 def a_star(graph, start, goal, heuristic):
4     visited = set()
5     pq = [(heuristic[start], 0, start, [start])]
6     exploration_order = []
7
8     while pq:
9         f, g, node, path = heapq.heappop(pq)
10
11         if node in visited:
12             continue
```

```

13         visited.add(node)
14         exploration_order.append(node)
15
16         if node == goal:
17             return path, g, exploration_order
18
19         for neighbor, edge_cost in sorted(graph[node].items()):
20             if neighbor not in visited:
21                 new_g = g + edge_cost
22                 new_f = new_g + heuristic[neighbor]
23                 heapq.heappush(pq,
24                               (new_f, new_g, neighbor, path + [neighbor])
25
26     return None, float('inf'), exploration_order

```

Listing 11: A* Search

8.4 Output

```

=====
6. A* SEARCH
=====
Path Found: Arad -> Sibiu -> RimnicuVilcea -> Pitesti ->
           Bucharest
Path Cost:  418
Hops:      4
Nodes Explored: ['Arad', 'Sibiu', 'RimnicuVilcea',
                'Fagaras', 'Pitesti', 'Bucharest']

```

Listing 12: A* output

8.5 Discussion

A* found the optimal path $\text{Arad} \rightarrow \text{Sibiu} \rightarrow \text{RimnicuVilcea} \rightarrow \text{Pitesti} \rightarrow \text{Bucharest}$ with a cost of **418**. Unlike Greedy Best-First (which chose Fagaras because $h = 176$), A* also considers the actual cost $g(n)$. From Sibiu, A* chose RimnicuVilcea because $f(\text{RimnicuVilcea}) = g(220) + h(193) = 413$, which is less than $f(\text{Fagaras}) = g(239) + h(176) = 415$. This allowed A* to discover the cheaper route through Pitesti. A* explored 6 nodes — slightly more than Greedy (4) but far fewer than UCS (13), demonstrating that the heuristic **guides the search efficiently while still guaranteeing optimality**.

9 Program 7 — Comparison of All Search Algorithms

9.1 Problem Statement

Compare all six search algorithms on the Romanian Map problem, highlighting their differences in path found, path cost, number of hops, and number of nodes explored.

9.2 Output

```
=====
COMPARISON OF ALL SEARCH ALGORITHMS
=====
```

Algorithm	Path	Cost
Hops		

BFS	Arad -> Sibiu -> Fagaras -> Bucharest	450
	3	
UCS	Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest	418
	4	
DFS	Arad -> Sibiu -> Fagaras -> Bucharest	450
	3	
IDDFS	Arad -> Sibiu -> Fagaras -> Bucharest	450
	3	
Greedy Best-First	Arad -> Sibiu -> Fagaras -> Bucharest	450
	3	
A*	Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest	418
	4	

Listing 13: Comparison of all algorithms

9.3 Discussion

Table 3: Comparison of All Search Algorithms

Algorithm	Path Cost	Optimal?	Nodes plored	Ex-	Type
BFS	450	No	9		Uninformed
UCS	418	Yes	13		Uninformed
DFS	450	No	4		Uninformed
IDDFS	450	No	—		Uninformed
Greedy Best-First	450	No	4		Informed
A*	418	Yes	6		Informed

Key observations:

1. **Only UCS and A* found the optimal path** (cost 418). Both guarantee optimality under their respective conditions.
2. **BFS, DFS, IDDFS, and Greedy** all found the suboptimal path (cost 450). BFS and IDDFS find the shortest-hop path but not the cheapest when step costs differ. DFS and Greedy have no optimality guarantee.
3. **DFS and Greedy explored the fewest nodes** (4 each), but at the cost of optimality.
4. **A* explored only 6 nodes** to find the optimal path — much fewer than UCS's 13 — showing that a good heuristic dramatically reduces the search effort while preserving optimality.
5. **UCS explored the most nodes** (13) because it has no heuristic guidance and must expand all cheaper partial paths before confirming the cheapest route.

10 Conclusion

This lab demonstrated the implementation and comparison of six fundamental search strategies in Artificial Intelligence on the Romanian Map problem:

1. **Breadth-First Search (BFS)** finds the shortest-hop path but is not cost-optimal when step costs vary. It requires significant memory ($O(b^d)$) as all nodes at each depth level must be stored.
2. **Uniform-Cost Search (UCS)** guarantees the cheapest path for non-negative edge costs by always expanding the lowest-cost node. It is optimal but may explore many nodes without heuristic guidance.
3. **Depth-First Search (DFS)** is memory-efficient ($O(bm)$) but neither complete nor optimal. Its results depend on the order of neighbour exploration.
4. **Iterative-Deepening DFS (IDDFS)** combines the completeness of BFS with the memory efficiency of DFS. It finds the shortest-depth path and is the preferred uninformed search when memory is limited.
5. **Greedy Best-First Search** uses a heuristic to guide the search towards the goal quickly, exploring very few nodes. However, it sacrifices optimality — the path found may be more expensive than the optimal route.
6. **A* Search** combines the actual cost $g(n)$ with the heuristic $h(n)$ to efficiently find the optimal path. With an admissible heuristic (straight-line distance in our case), A* guarantees optimality while exploring far fewer nodes than UCS.

The Romanian Map problem clearly illustrates that **informed search (A*)** provides the best trade-off between efficiency and optimality when a good heuristic is available, while **uninformed search (UCS)** guarantees optimality at the cost of exploring more nodes. The choice of search strategy depends on the problem characteristics, available heuristic information, and the relative importance of optimality versus computational efficiency.